



भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology Hyderabad

GPU Programming

**Parallel BFS implementation using CUDA with overlapped
computation and data transfer**

Name - Deeba Afridi

Contents

1	Introduction	1
2	Algorithmic Design and Implementation Strategy	1
2.1	Graph Representation	1
2.2	Baseline BFS Design	1
2.3	Stream-Optimized BFS Design	2
2.4	CUDA Graph optimized BFS Design	2
2.5	Kernel Summary	2
3	Performance Analysis	3
3.1	Baseline BFS Results	3
3.2	Streamed BFS Results	4
3.3	CUDA Graphs BFS Results	7
4	Speedup and Comparison	9
4.1	Speedup of stream optimization over baseline	9
4.2	Speedup of CUDA graph optimization over baseline	10
4.3	Comparison in the timing of Baseline, Stream Optimization, and CUDA Graph Optimization	12
5	Performance Discussion	14
6	Experimental Setup	14
7	Conclusion and Future Work	15

Abstract

This report presents the design, implementation, and performance evaluation of a parallel Breadth-First Search (BFS) traversal over a KNN graph using CUDA.

It consists of three implementations (tasks) as part of the assignment:

- (1) a baseline CUDA BFS without overlap,
- (2) a stream-optimized version that overlaps host-device transfers with GPU computations, and
- (3) a CUDA Graph-based implementation that captures and replays the entire kernel launch sequence to significantly reduce CPU-side launch overheads.

This report discusses the design choices, kernel structure, memory layout, and pinned-memory optimizations used in these implementations. Performance is evaluated across multiple KNN graphs of varying sizes. BFS discovery time is measured across several runs to compute the average runtime and speedup. The results show the benefits of overlapping data transfer with computation using CUDA streams, and further reducing launch overhead using CUDA Graphs, particularly for deeper BFS levels and small kernel workloads.

1 Introduction

Breadth-First Search (BFS) is a fundamental graph traversal algorithm widely used in many GPU-accelerated workloads. In real-world scenarios involving large datasets, entire graphs cannot be stored in GPU memory and must be streamed from host memory, making PCIe transfers a major bottleneck.

The goal of this assignment is to implement BFS using :

1. a baseline CUDA approach without overlap,
2. a stream-optimized variant that overlaps computation and data transfer, and
3. a CUDA Graph optimization for capturing and executing the kernel launch sequence

This report presents detailed analysis for Task 1 (Baseline BFS) and Task 2 (Streamed BFS using CUDA Streams) and Task 3 (BFS with CUDA graphs).

2 Algorithmic Design and Implementation Strategy

2.1 Graph Representation

The graph is a KNN adjacency list stored in CSV format. Each row contains the node ID followed by its R neighbors. The graphs used in this evaluation are combinations of:

- Nodes: 100 , 50K, 100K, 250K, 500K
- Degree (R): 8 , 64, 128
- Feature dimension: 128

The adjacency list is flattened into a 1D array of size `nodeCount` $\times$ `neighborCount`. Node feature vectors (128-dimensional SIFT) are loaded from a `.fvecs` file.

2.2 Baseline BFS Design

The baseline implementation uses a level-synchronous BFS:

1. Maintain a host-side frontier vector.
2. For each node in the frontier, gather all neighbor lists into a contiguous adjacency block.
3. Copy this block synchronously to the GPU.
4. Launch CUDA kernels:
 - `bfsProcessFrontier`: marks newly discovered nodes using `atomicCAS`.
 - `computeDistances`: computes Euclidean distance of each frontier node from the start node.
5. Copy next-frontier flags and distances back to the host.
6. Construct next frontier and repeat until empty.

All memory transfers and kernel launches are synchronous, resulting in GPU idle periods.

2.3 Stream-Optimized BFS Design

To hide PCIe transfer latency, the streamed version uses:

- **Pinned (page-locked) host memory** for adjacency blocks, frontier vectors, distances, and next flags.
- **Multiple CUDA streams** ($N = 4$) to process different frontier batches simultaneously.
- **Asynchronous host-device transfers** using `cudaMemcpyAsync`.
- **Batching**: frontier is divided into $\lceil \frac{\text{frontierCount}}{\text{NUM_STREAMS}} \rceil$ -sized blocks.

For each stream:

1. Copy adjacency and vector batch to device asynchronously.
2. Launch BFS and distance kernels.
3. Copy results back asynchronously.

Finally, the host synchronizes all streams, merges next-frontier flags, and proceeds to the next level.

2.4 CUDA Graph optimized BFS Design

To reduce the overhead of repeated kernel launches by CPU and capture the entire kernel launch sequence:

- BFS processing for each level is captured inside a graph.
 1. H2D copy of adjacency block,
 2. H2D copy of frontier vectors,
 3. BFS kernel launch,
 4. Distance kernel launch,
 5. D2H copy of results.
- The graph is instantiated once using `cudaGraphInstantiate`.
- For remaining BFS levels, only kernel arguments are updated.

This reduces:

- Kernel-launch overhead,
- CPU scheduling time,
- Per-level CUDA API calls.

2.5 Kernel Summary

BFS Kernel : Each thread handles one frontier node and scans its R neighbors:

`atomicCAS(visited[neighbor], 0, 1)`

ensures correctness under concurrent updates.

Distance Kernel : Computes:

$$d = \sqrt{\sum_{i=1}^{dim} (x_i - s_i)^2}$$

for each frontier node.

Both kernels launch with 256 threads/block.

3 Performance Analysis

3.1 Baseline BFS Results

Dataset	Nodes	Degree	Start Node	Average Time of 5 runs(ms)
graph_100_8.csv	100	8	95	2.396
graph_5k_128.csv	5,000	128	4857	11.693
graph_50k_64.csv	50,000	64	43525	64.646
graph_100k_128.csv	100,000	128	42202	189.76
graph_100k_64.csv	100,000	64	41915	130.202
graph_250k_64.csv	250,000	64	41915	331.361
graph_500k_64.csv	500,000	64	41915	659.270

Table 1: Average BFS discovery times of 5 runs for Task 1 (Baseline with no overlap) across all datasets.

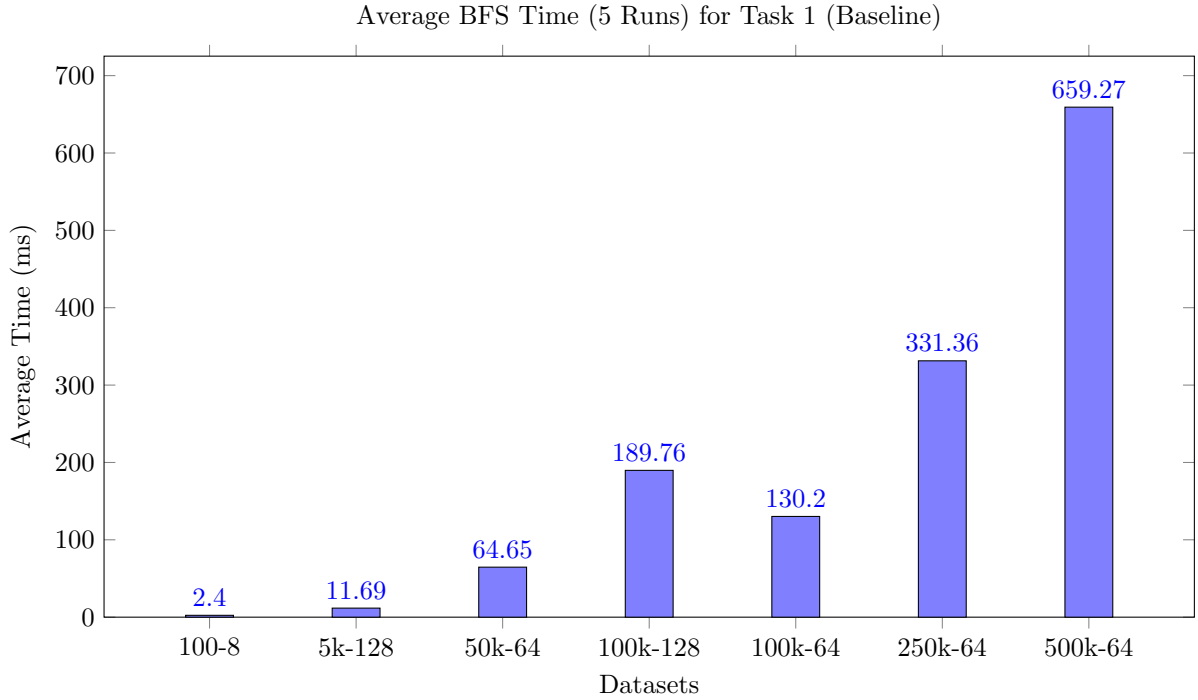


Figure 1: Average BFS discovery times (5 runs) for Task 1 across all datasets.

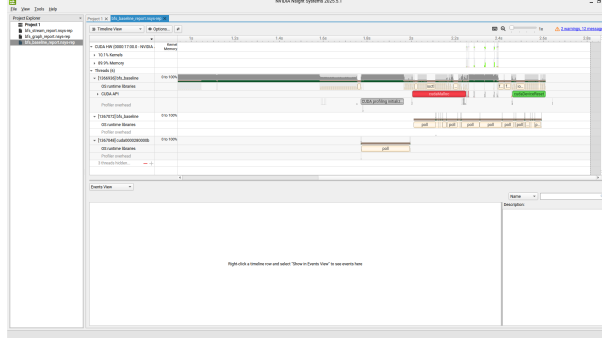


Figure 2: nsys report of baseline

Observations from baseline numbers:

- **Small graph (graph_100_8.csv):** Runtime is very small (2.396 ms) and dominated by fixed overheads (kernel launch, memory operations).
- **Moderate graph (graph_5k_128.csv):** BFS discovery time is 11.693 ms. This increase from 100 nodes demonstrates that computation begins to dominate even at this scale. The per-node work is higher due to the larger degree (128 neighbors) and feature vector distance computation.
- **Medium graphs (50k, 100k):** Moving to 50k nodes (degree 64) increases BFS time to ~ 64.6 ms; for 100k nodes (degree 128) it rises to ~ 189.7 ms. This reflects larger frontiers, neighbor expansion, and Euclidean distance calculations.
- **Large graphs (250k, 500k):** Runtime scales approximately linearly with node count for fixed degree ($R=64$), indicating that neighbor traversal and distance computation dominate cost.
- **Stability:** Across repeated runs, the BFS times vary minimally (e.g., 11.615–11.801 ms for 5k graph; 187–190 ms for 100k graph), showing consistent GPU performance.

3.2 Streamed BFS Results

Dataset	Nodes	Degree	Start Node	Average Time of 5 runs(ms)
graph_100_8.csv	100	8	95	2.326
graph_5k_128.csv	5,000	128	4857	12.873
graph_100k_128.csv	100,000	128	42202	97.899
graph_50k_64.csv	50,000	64	43525	45.535
graph_100k_64.csv	100,000	64	41915	81.896
graph_250k_64.csv	250,000	64	41915	197.888
graph_500k_64.csv	500,000	64	41915	389.220

Table 2: Average BFS discovery times of 5 runs for Task 2 (CUDA Streams) across all datasets.

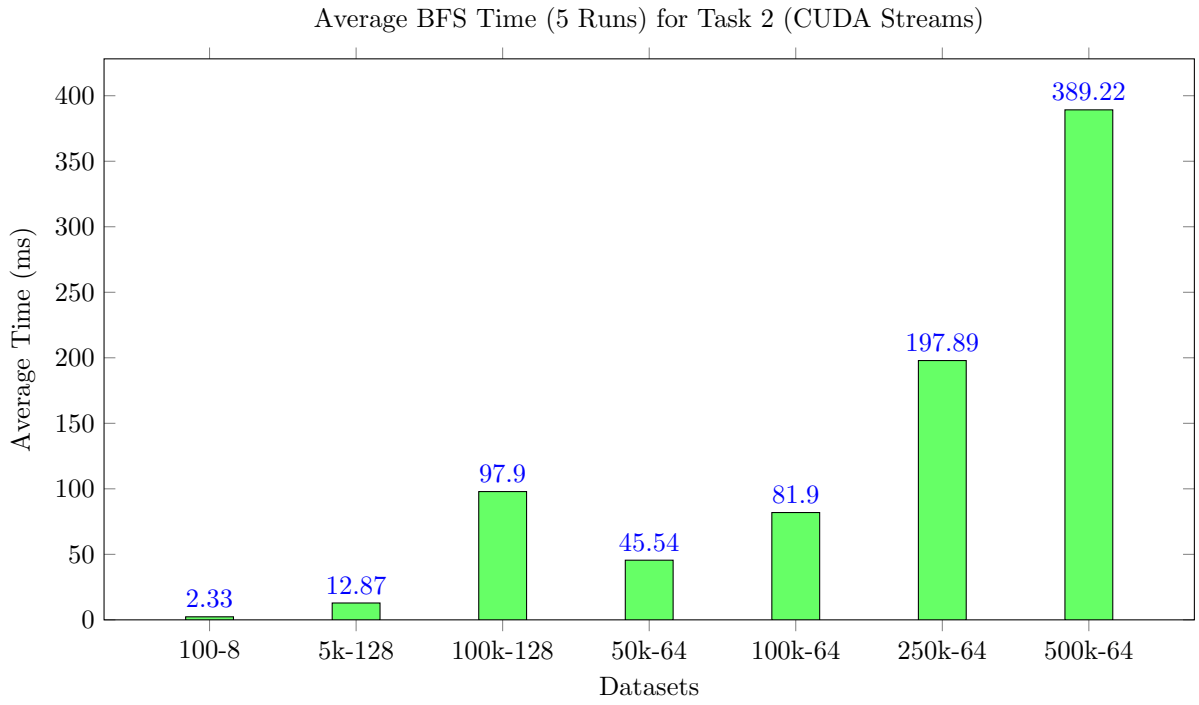


Figure 3: Average BFS discovery times (5 runs) for Task 2 (CUDA Streams).

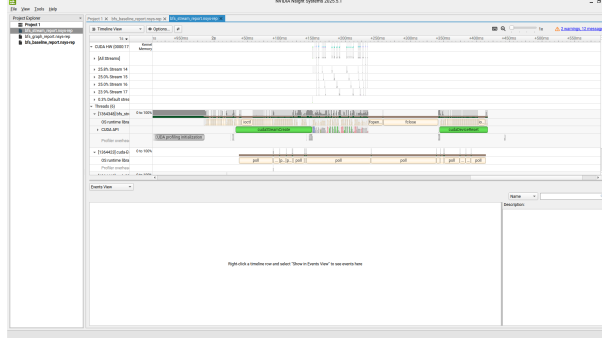


Figure 4: nsys report of stream optimization

Observations from streamed numbers:

- **Small graph (graph_100_8.csv):** Streams yield only a slight improvement (2.396 ms $\rightarrow$ 2.326 ms, about 3%) as compared to the baseline which is nearly negligible, which is expected because the total data transferred and per-level compute are tiny. Stream management overhead and kernel-launch costs are comparable to the actual work for such small inputs. For graph_100_8.csv, streams even did not show the improvement as for smaller graphs streams create an overhead.
- **Medium graphs:** Streams show substantial improvements. For graph_50k_64, average time drops from 64.646 ms to 45.535 ms; for 100k it drops from 130.202 ms to 81.896 ms as compared to the baseline. These reductions indicate that asynchronous copies and overlapping transfer with kernel work effectively hide PCIe latency and improve device utilization when per-level work is large.
- **Large graphs:** For 250k and 500k node graphs, streams reduce runtime significantly (331.361 ms $\rightarrow$ 197.888 ms and 659.270 ms $\rightarrow$ 389.220 ms respectively) from baseline. This suggests that for large frontier sizes the overlap between memcopy and kernel execution, plus improved concurrency between stream queues, produces large end-to-end gains.
- **Stability:** Small variations across runs indicate stable performance despite asynchronous execution.
- **Conclusion:** As the graph size increases, streamed BFS increasingly outperforms baseline. Large frontiers help hide PCIe latency.

3.3 CUDA Graphs BFS Results

Dataset	Nodes	Degree	Start Node	Average Time of 5 runs(ms)
graph_100_8.csv	100	8	95	7.7011
graph_50k_64.csv	50,000	64	43525	48.533
graph_100k_64.csv	100,000	64	41915	85.146
graph_250k_64.csv	250,000	64	41915	208.819
graph_500k_64.csv	500,000	64	41915	404.243
graph_5k_128.csv	5,000	128	4857	13.924
graph_100k_128.csv	100,000	128	42202	106.523

Table 3: Average BFS discovery times of 5 runs for Task 3 (CUDA Graphs) across all datasets.

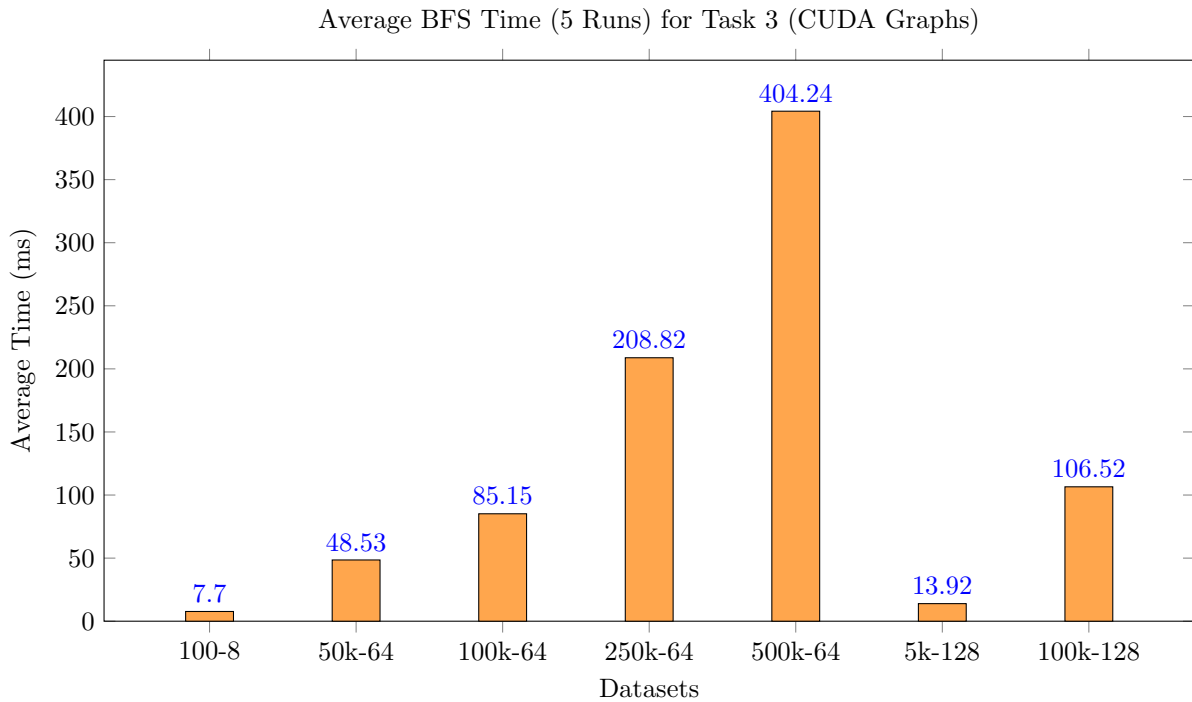


Figure 5: Average BFS discovery times (5 runs) for Task 3 (CUDA Graphs).

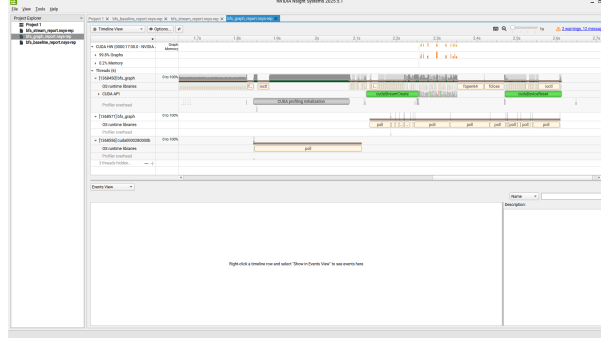


Figure 6: nsys report of graph optimization

Observations from CUDA Graph numbers:

- **Small graph (graph_100.8.csv):** CUDA Graphs perform significantly worse than the baseline (2.396 ms $\rightarrow$ 7.701 ms). This is expected because kernel launch overhead dominates execution time on tiny inputs. CUDA Graph construction, instantiation, and replay introduce fixed costs that overshadow the small amount of actual BFS work. Thus, CUDA Graphs are not beneficial for very small workloads.
- **Medium graphs:** For graphs of size 50k–100k with degree 64, CUDA Graphs show noticeable improvements over baseline (64.646 ms $\rightarrow$ 48.533 ms and 130.202 ms $\rightarrow$ 85.146 ms). The benefit comes from reduced launch overhead per BFS level and better batching of repeated GPU work. Since BFS executes many small kernels per level, CUDA Graphs reduce per-level scheduling overhead and improve runtime determinism.
- **Large graphs:** The performance gains continue at larger scales. For 250k and 500k node graphs, runtime reduces substantially (331.361 ms $\rightarrow$ 208.819 ms and 659.270 ms $\rightarrow$ 404.243 ms). At these sizes, the fixed cost of graph capture is paid off over many BFS iterations, and CUDA Graph replays become highly efficient. This demonstrates the strength of CUDA Graphs for workloads with repetitive kernel patterns.
- **High-degree graphs (128-degree cases):** For graph_5k_128, performance is slightly worse than baseline (12.873 ms $\rightarrow$ 13.924 ms). The graph is small enough that CUDA Graph overhead outweighs the benefit. However, for graph_100k_128, CUDA Graphs provide a major improvement (189.76 ms $\rightarrow$ 106.523 ms), showing that once the workload becomes large and structured, CUDA Graphs effectively reduce kernel launch overhead.
- **Stability:** The CUDA Graph results show very little variation across repeated runs, which highlights how consistently and predictably CUDA Graphs execute once the graph is captured.
- **Conclusion:** CUDA Graph optimization becomes increasingly effective as graph size and BFS traversal work increase. While small graphs do not benefit, medium and large graphs show consistent and substantial speedups due to reduced kernel launch overhead, batched execution, and improved scheduling efficiency.

4 Speedup and Comparison

4.1 Speedup of stream optimization over baseline

Dataset	Baseline (ms)	Streams (ms)	Speedup (vs Baseline)
graph_100_8.csv	2.396	2.326	1.03×
graph_50k_64.csv	64.646	45.536	1.42×
graph_100k_64.csv	130.202	81.895	1.59×
graph_250k_64.csv	331.361	197.888	1.67×
graph_500k_64.csv	659.270	389.219	1.69×
graph_5k_128.csv	11.693	12.873	0.91×
graph_100k_128.csv	189.76	97.899	1.94×

Table 4: Comparison of Baseline BFS and Task 2 (Stream optimization) with computed speedups.

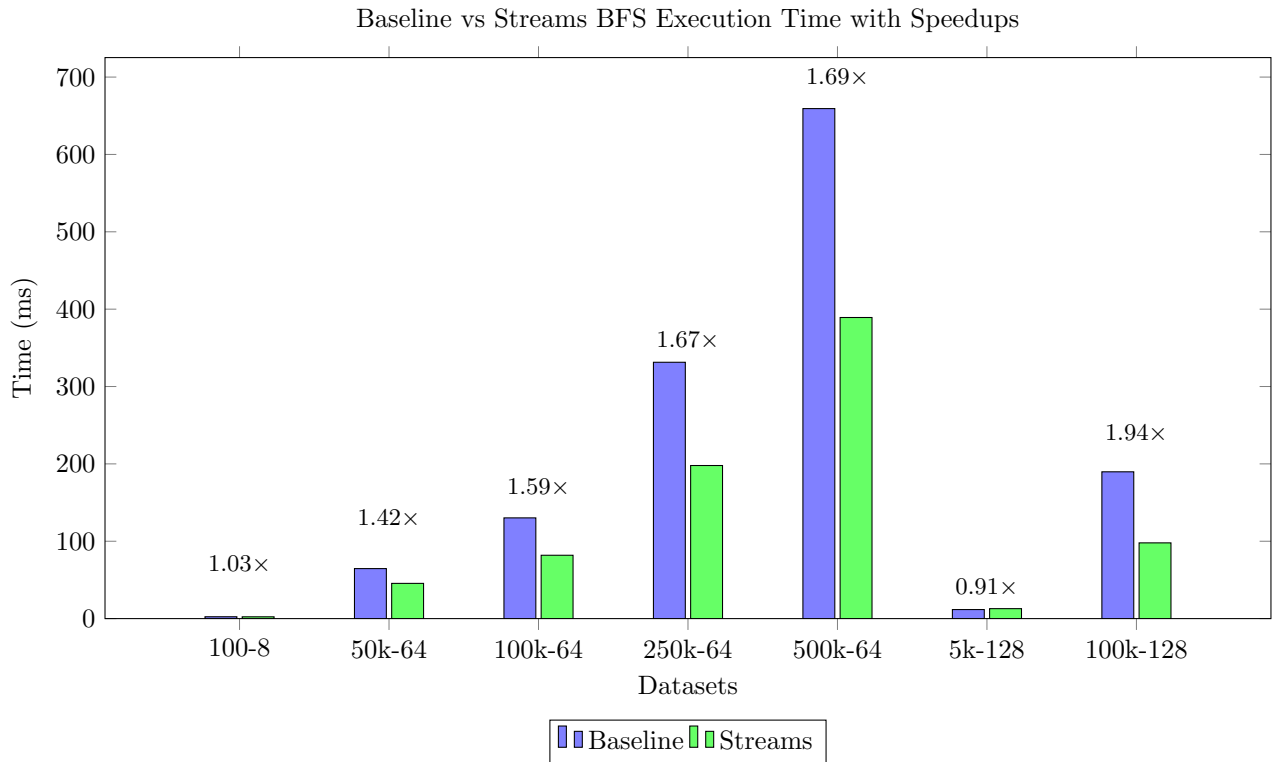


Figure 7: Baseline BFS vs CUDA Streams performance with only speedup labels.

Insights on speedup trends:

- **Increasing speedup with scale:** The speedup of stream optimization over baseline grows with dataset size, from $1.03\times$ (100 nodes) to about $1.69\times$ (500k nodes). This is because the ratio of transferable data and kernel compute increases with graph size, creating more opportunity to hide transfer latency behind computation.
- **Overlap effectiveness:** For larger graphs the time spent moving adjacency blocks and frontier vectors is substantial; overlapping these transfers with independent kernel execution reduces effective transfer time, leading to larger overall savings.
- **When Streams help most:** Streams are particularly effective when:
 - per-level frontiers are large (many nodes),
 - neighbor degree R is large (more per-node work),

4.2 Speedup of CUDA graph optimization over baseline

Dataset	Baseline (ms)	CUDA Graphs (ms)	Speedup
graph_100_8.csv	2.396	7.701	$0.31\times$
graph_50k_64.csv	64.646	48.533	$1.33\times$
graph_100k_64.csv	130.202	85.146	$1.53\times$
graph_250k_64.csv	331.361	208.819	$1.59\times$
graph_500k_64.csv	659.270	404.243	$1.63\times$
graph_5k_128.csv	12.873	13.924	$0.92\times$
graph_100k_128.csv	189.760	106.523	$1.78\times$

Table 5: Comparison of Baseline BFS and CUDA Graphs optimization with speedups.

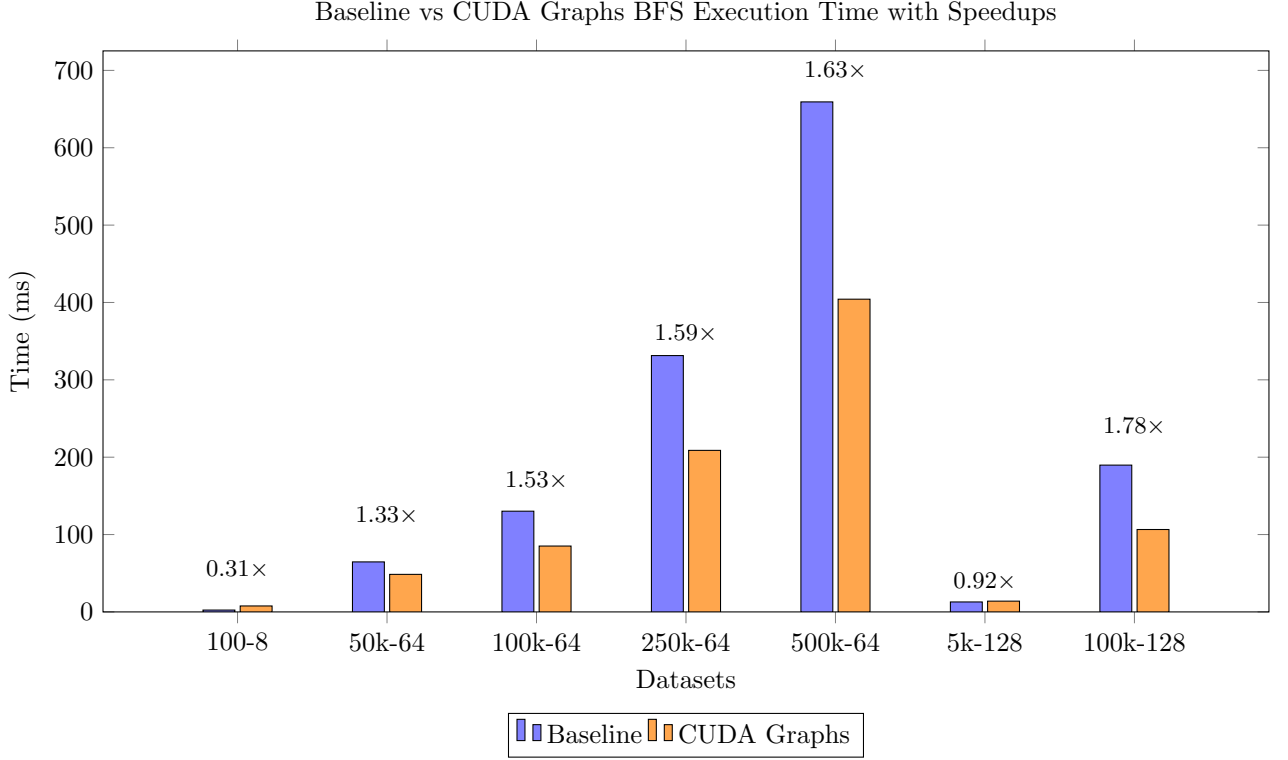


Figure 8: Baseline BFS vs CUDA Graphs performance with only speedup labels.

Insights on CUDA Graph speedup trends:

- **Reduced benefit for very small graphs:** For tiny inputs (e.g., graph_100.8), CUDA Graphs actually slow down execution ($0.31\times$ speedup). This is expected because the cost of capturing and instantiating the CUDA Graph dominates the extremely small amount of actual work.
- **Consistent gains for medium and large graphs:** Once the graph size becomes meaningful (50k nodes and above), CUDA Graphs provide steady improvements ranging from $1.3\times$ to nearly $1.8\times$. This is because CUDA Graphs remove repeated kernel-launch overhead inside the BFS loop, allowing the GPU to run kernels back-to-back with minimal CPU intervention.
- **More effective when BFS has many levels:** Since BFS requires launching multiple kernels per level, graphs with more layers benefit more—the fixed scheduling overhead saved per level accumulates into larger total speedups.
- **Where CUDA Graphs help the most:**
 - workloads with many small or moderate kernels per iteration,
 - large graphs that result in deep BFS traversals,
 - cases where CPU launch overhead becomes comparable to kernel execution time.

4.3 Comparison in the timing of Baseline, Stream Optimization, and CUDA Graph Optimization

Dataset	Baseline (ms)	Streams (ms)	CUDA Graphs (ms)
graph_100_8.csv	2.396	2.326	7.701
graph_5k_128.csv	11.693	12.873	13.924
graph_50k_64.csv	64.646	45.535	48.533
graph_100k_64.csv	130.202	81.896	85.146
graph_100k_128.csv	189.760	97.899	106.523
graph_250k_64.csv	331.361	197.888	208.819
graph_500k_64.csv	659.270	389.220	404.243

Table 6: Side-by-side comparison of runtime for Baseline BFS, CUDA Streams optimization, and CUDA Graphs optimization across all datasets.

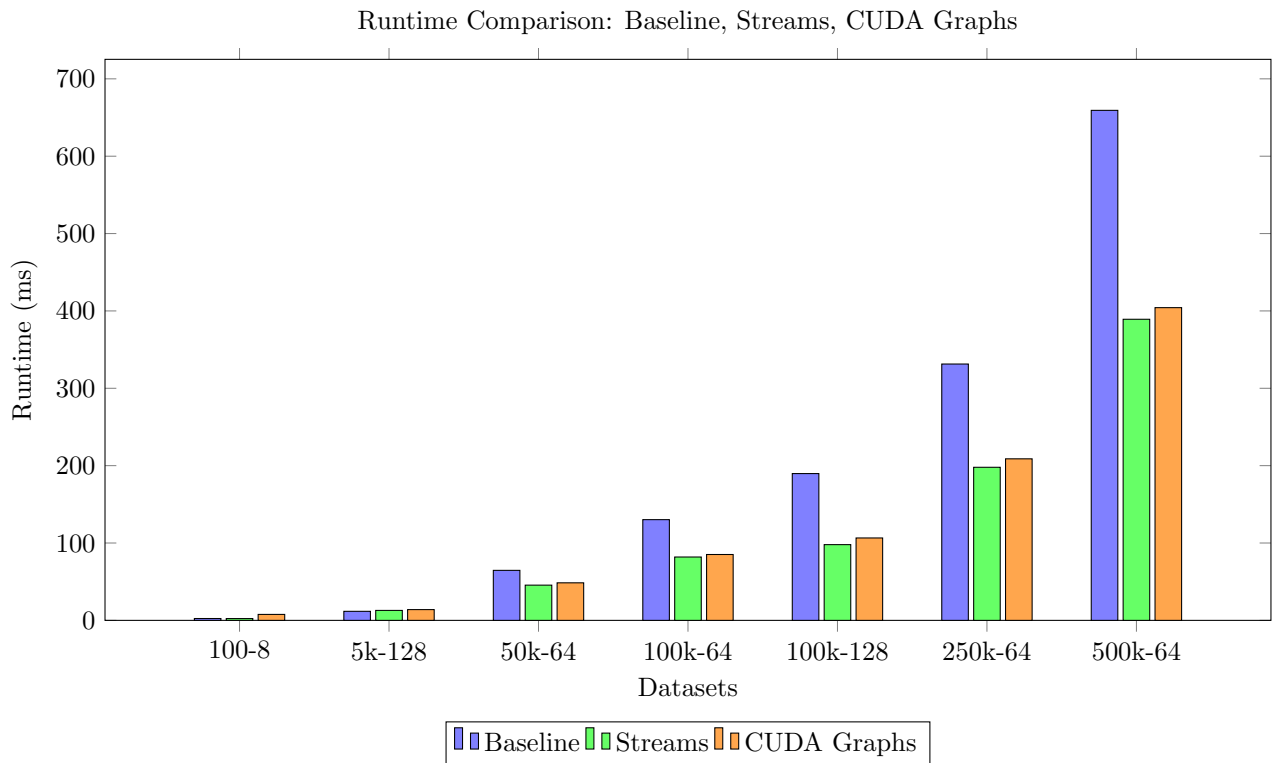


Figure 9: Runtime comparison of Baseline, Streams, and CUDA Graphs across datasets.

Insights from the comparison:

- **Streams consistently outperform the baseline for medium and large graphs.** Across 50k, 100k, 250k, and 500k node graphs, stream optimization provides a noticeable reduction in runtime compared to the baseline. This confirms that overlapping data movement with compute becomes increasingly effective as graph size and frontier widths grow.
- **CUDA Graphs also improve performance for large datasets, but not as much as streams.** For all large graphs (50k and above), CUDA Graphs reduce execution time compared to the baseline, but the improvement is slightly smaller than what streams achieve. This is expected because BFS involves level-by-level irregular workloads, where CUDA Graph replay cannot fully eliminate kernel-launch overheads or synchronize away frontier dependencies.
- **Both optimizations underperform on very small graphs.** For `graph_100_8.csv`, streams give only a negligible improvement, and CUDA Graphs are actually slower than the baseline. This happens because the overhead of stream management or graph capture dominates the extremely small amount of work.
- **High-degree small graphs (e.g., 5k, 128-degree) show limited benefit.** In `graph_5k_128.csv`, both optimizations are slightly slower than the baseline. The combination of small graph size and high per-node degree means the kernels themselves are small and fast; in such cases, additional overhead outweighs any benefit from overlap.
- **Best cases occur for very large graphs.** At 250k and 500k nodes, both streams and CUDA Graphs show clear improvements:
 - Streams reduce time by about 40%
 - CUDA Graphs reduce time by about 35%

The large frontier sizes and repeated memory operations provide enough parallelism for these optimizations to hide latency and reduce overhead.

- **Overall trend: Streams > CUDA Graphs > Baseline (for large graphs).** For most medium and large datasets, streams consistently perform the best, CUDA Graphs follow closely, and the baseline is the slowest. The relative ranking flips only for small graphs, where simple execution is faster than any optimization overhead.

5 Performance Discussion

- **Dominant cost components:** For small graphs, fixed overheads (launch, host setup) dominate. For medium and large graphs, device compute and memory-bandwidth bound neighbor traversal and distance computation dominate.
- **Why streams help:** Streams reduce the visible transfer cost by overlapping host-to-device/device-to-host copies with kernel execution. For the larger graphs, the relative reduction in end-to-end time is significant because kernels take longer to run and thus provide a larger window to hide transfers.
- **Why CUDA Graphs will help:** CUDA Graphs can further reduce CPU scheduling overheads (kernel launch and launch synchronization costs). This will benefit medium-sized frontiers that launch many kernels (per-level launches) and smaller frontiers where CPU overhead is proportionally larger. Combining CUDA Graphs with streams and pinned memory gives the most complete optimization.

6 Experimental Setup

The experiments were conducted on a GPU server equipped with the following hardware and software configuration:

- **GPU Model:** NVIDIA A100 80GB PCIe
- **GPU Memory:** 80 GB
- **CUDA Runtime Version:** 13.0
- **CUDA Toolkit Version:** 12.0
- **NVIDIA Driver Version:** 580.82.07
- **Operating System:** Linux (GPU Server)
- **Datasets:** KNN graphs of various sizes, e.g., `graph_100_8.csv`, `graph_50k_64.csv`, `graph_100k_64.csv`, `graph_100k_128.csv`, `graph_250k_64.csv`, `graph_500k_64.csv`, `graph_5k_128.csv`
- **Feature Vectors:** 128-dimensional SIFT vectors loaded from `sift_base.fvecs`
- **Measurement Method:** Each experiment is repeated 5 times to obtain stable average runtime.

7 Conclusion and Future Work

In this report, we implemented and analyzed three versions of the Breadth-First Search (BFS) algorithm on the GPU using CUDA: the baseline version (without any overlap), a stream-optimized version, and a CUDA Graph-based version. Our goal was to understand how different forms of GPU execution optimization impact BFS performance, especially given its irregular memory-access pattern and per-level workload variability.

The report presents implementation and analysis on numbers of baseline BFS (without overlap), stream-optimized BFS and CUDA graph optimized BFS implementations using CUDA.

Overall, our findings indicate that:

- **Streams provide the best performance gains for large datasets** by effectively hiding PCIe transfer latency behind computation.
- **CUDA Graphs are beneficial when BFS involves many small kernel launches**, as they significantly reduce CPU-side overhead.
- **Baseline BFS performs reasonably well only for tiny graphs**, where optimization overhead dominates.

Future Work

- Using dynamic parallelism to allow the GPU to manage BFS expansion autonomously.
- Using device-side frontier compaction to reduce host transfers.
- Exploring warp-specialized kernels for neighbor expansion.